

LABORATORY ASSIGNMENT

Course:	Full Stack Web Development
Course Code:	23CM4121
Experiment No.:	1
Student Name:	Appala reddy.k
Roll No.:	A24126552089
Date:	15-09-2026

1. TITLE

JavaScript Multilevel Inheritance Demonstration

2. AIM

To demonstrate object-oriented programming in JavaScript by creating ES6 classes and implementing multilevel inheritance with inherited and class-specific methods.

3. OBJECTIVE

The objectives of this experiment are:

- To create reusable object-oriented structures using ES6 classes.
- To implement inheritance between parent, child, and grandchild classes.
- To use the `super()` constructor to initialize inherited properties.
- To execute inherited and specialized methods and display the results through DOM manipulation.

4. THEORY

Object-Oriented Programming (OOP) is a programming paradigm in which data and behavior are organized into objects. JavaScript supports OOP using prototypes, while ES6 classes provide a cleaner syntax for defining constructors and methods. Inheritance allows a derived class to reuse properties and methods from a parent class. With the `extends` keyword, a child class can inherit functionality and also add its own behavior. The `super()` keyword is used to invoke the parent constructor.

5. ALGORITHM / PROCEDURE

1. Create an HTML page containing a styled output container.
2. Define a base Device class with a `deviceName` property and a `powerOn` method.
3. Define a Laptop class that extends Device and adds a `model` property.
4. Define a GamingLaptop class that extends Laptop and adds a `launchGame` method.
5. Create a GamingLaptop object with sample device, model, and game values.
6. Use `getElementById()` to select the output container.
7. Display the results of inherited and specialized methods in the webpage.

6. PROGRAM

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```

<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>JS Multilevel Inheritance</title>
<style>
  body {
    font-family: Arial, sans-serif;
    padding: 20px;
    background-color: #f4f4f9;
  }
  .output-box {
    background-color: white;
    border: 2px solid #007bff;
    border-radius: 8px;
    padding: 15px;
    width: 400px;
  }
  p { font-size: 18px; color: #333; margin: 10px 0; }
</style>
</head>
<body>
  <h2>JavaScript Multilevel Inheritance Demo</h2>
  <div class="output-box" id="displayArea"></div>

  <script>
    class Device {
      constructor(deviceName) {
        this.name = deviceName;
      }

      powerOn() {
        return `The ${this.name} device is powered on.`;
      }
    }

    class Laptop extends Device {
      constructor(deviceName, modelName) {
        super(deviceName);
        this.model = modelName;
      }

      showModel() {
        return `Model: ${this.name} ${this.model}`;
      }
    }

    class GamingLaptop extends Laptop {
      constructor(deviceName, modelName, gameName) {
        super(deviceName, modelName);
        this.game = gameName;
      }

      launchGame() {
        return `${this.name} ${this.model} is launching ${this.game}.`;
      }
    }

    const myLaptop = new GamingLaptop("Acer", "Nitro V", "Valorant");
    const display = document.getElementById("displayArea");

    display.innerHTML = `
      <p><b>Parent Method:</b> ${myLaptop.powerOn()}</p>
      <p><b>Inherited Method:</b> ${myLaptop.showModel()}</p>
      <p><b>Child Method:</b> ${myLaptop.launchGame()}</p>
    `;
  </script>
</body>
</html>

```

7. CODE EXPLANATION

Code / Syntax	Explanation
---------------	-------------

class Device {}	Base/Parent class containing the common device property and powerOn method.
class Laptop extends Device	Child class that inherits functionality from Device.
super(deviceName)	Calls the parent constructor to initialize the device name.
class GamingLaptop extends Laptop	Grandchild class demonstrating multilevel inheritance.
myLaptop.powerOn()	Executes the inherited method from the Device class.
myLaptop.showModel()	Executes the inherited method from the Laptop class.
myLaptop.launchGame()	Executes the method defined specifically in GamingLaptop.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Create GamingLaptop object	Object initializes with device, model and game	Object created successfully	Pass
TC-02	Call powerOn()	Parent method returns device power message	Correct power message displayed	Pass
TC-03	Call showModel()	Inherited Laptop method returns model details	Correct model details displayed	Pass
TC-04	Call launchGame()	GamingLaptop method returns game launch message	Correct game message displayed	Pass
TC-05	Render results in DOM	All three messages appear in output box	All messages displayed correctly	Pass

9. OUTPUT

JavaScript Multilevel Inheritance Demo

Parent Method: The Acer device is powered on.

Inherited Method: Model: Acer Nitro V

Child Method: Acer Nitro V is launching Valorant.

10. RESULT

Thus, the JavaScript multilevel inheritance demonstration was successfully executed. The inherited and child-class methods produced the expected output and were rendered successfully on the webpage.